

# 2026 年全国大学生电子设计竞赛 广东赛区竞赛

## 车载平衡滚球运动控制系统（H 题）



2026 年 8 月 1 日

**摘要：**本系统面向车载平衡滚球控制任务，由循线底盘、视觉图传、水管倾角执行、显示和电源模块组成。底盘以 MSPM0G3507 为核心，采用 MG513 直流减速电机、TB6612、500 线 GMR 编码器和 8 路灰度传感器，实现循线、速度闭环和定点停车。MaixCAM2 俯视检测钢球，经标定得到钢球相对的位置和速度，并将数据发送至主控；视频经 WebRTC 推送至赛道外平板，由外部设备通过浏览器实时显示和录屏保存。主控采用速度提前预测误差 PID 控制水管倾角：将位置误差与钢球速度加权合成为预测误差，计算目标倾角并驱动 42 型闭环步进电机。系统设置视觉丢失、通信超时和行程限位保护。经实测，系统各项功能与指标均满足赛题要求。

**关键词：**MaixCAM2；闭环步进电机；预测误差 PID 控制

# 车载平衡滚球运动控制系统（H 题）

## 一、引言

本系统针对 2026 年全国大学生电子设计竞赛 H 题设计，要求小车自主完成环形赛道循线行驶，同时通过调节水管凹槽倾角控制钢球位置，实现静态定位、短距平衡与整圈行驶不掉落，并配套实时图传与视频录存功能。

系统以 TI MSPM0G3507 微控制器为核心，采用“平稳循线 + 主动平衡”一体化控制方案。底盘侧通过 8 路灰度传感器 + 双 GMR 编码器构成双闭环，经差速 PID 实现稳定循线、分段减速与定点停车，降低行驶惯性对钢球的扰动；视觉侧由 MaixCAM2 部署 YOLO11 模型识别钢球，经像素标定输出位置与速度信息；主控采用带速度前馈的预测 PID 算法求解目标倾角，驱动闭环步进电机与连杆机构调节水管俯仰，实现静态场景下的钢球平衡；同时通过 WebRTC 链路完成远程图传与同步录屏。

## 二、系统总体方案

系统由循线底盘、视觉检测与图传、水管倾角执行、显示、电源五大功能模块组成，整体架构如图 1 所示。系统以 MSPM0G3507 为核心构建“感知—决策—执行”闭环控制体系，共包含四条核心信号链路。

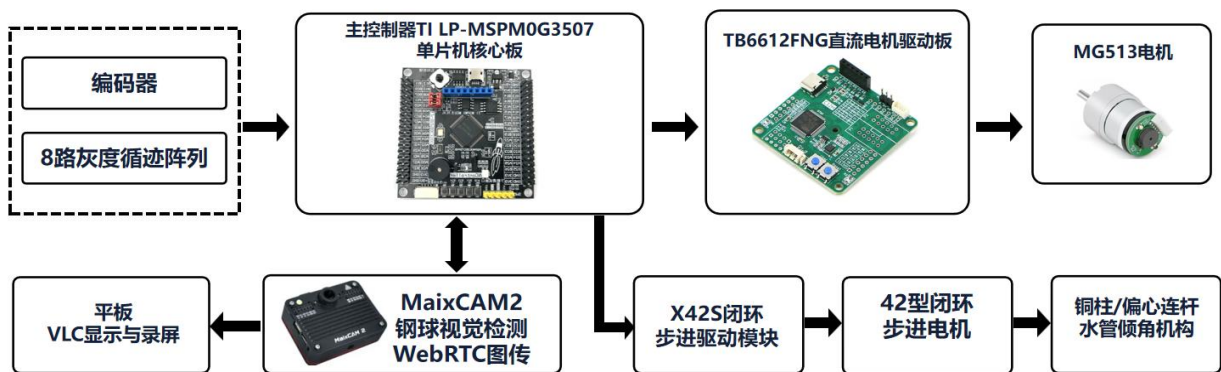


图 1 系统总体框图

### 1. 循线底盘信号链路

实现小车循线行驶、分段减速与定点停车。8 路灰度传感器实时检测黑线横向偏移量，经 GPIO 端口送入主控；双路 500 线 GMR 编码器采集左右轮转速与累计里程，通过定时器捕获端口输入。主控结合灰度偏差与速度反馈完成差速修正与速度闭环计算，输出 PWM 波与方向控制信号至 TB6612 驱动模块，驱动 MG513 直流减速电机运行。

### 2. 钢球视觉与图传信号链路

同步完成钢球检测与远程图传录屏。MaixCAM2 采用独立电源供电，俯视采集钢球画面，通过 YOLO11 模型识别目标并提取检测框中心坐标，经像素 - 距离标定换算为钢球实际位置  $x$  与运动速度  $v$ 。检测结果以约 50fps 帧率经 UART4 发送至主控；同时设备通过板载 WiFi，以 WebRTC 协议将实时画面推送至赛道外平板电脑，支持浏览器端查看与同步录屏。

### 3. 倾角执行信号链路

通过调节水管俯仰角度控制钢球运动。主控将钢球位置误差与速度加权合成为预测误差，经 PID 运算得到水管目标倾角，限幅后通过 UART1 发送至 X42S 闭环步进驱动器。驱动器驱动 42 型闭环步进电机，经偏心连杆机构将旋转运动转换为水管俯仰动作，角度调节范围约  $\pm 50^\circ$ ；电机自带 1000 线编码器反馈，实现角度全闭环控制。

### 4. 供电信号链路

以 12V 锂电池为总电源，采用分支路供电 + 公共地设计，各模块电源入口配置滤波去耦电容，保障供电稳定与参考电位一致。动力支路直接为 TB6612、X42S 驱动器供电；低压支路降压生成 5V 为灰度传感器供电，再经 LDO 降压至 3.3V 为主控与 OLED 显示模块供电；MaixCAM2 采用自带电源独立供电，不接入车载电源网络。

## 三、方案论证

本章围绕小车驱动与循线、钢球视觉检测与控制、水管倾角执行三个核心部分开展方案对比与选型。

### 1. 小车驱动与循线方案选型

方案一：纯灰度开环循线。通过 8 路灰度传感器偏移量直接控制差速，结构简单但无速度反馈，无法补偿电机差异与路面扰动，高速易偏航，难以实现精准减速与停车。

方案二：灰度循线+编码器速度闭环。8 路灰度传感器检测赛道偏移，双 GMR 编码器反馈左右轮转速与里程，主控通过增量式 PID 实现差速修正与速度闭环，抗干扰能力强，可精准实现分段减速与定点停车。

选用方案二。

### 2. 钢球视觉检测与控制方案选型

方案一：传统图像处理+单位置环控制。采用颜色阈值、霍夫圆检测等传统算法识别钢球，仅依据位置误差输出控制量。对光照、反光敏感，无速度补偿，高速运动下易振荡，且不具备图传功能。

方案二：YOLO 目标检测+预测 PID+无线图传。MaixCAM2 部署 YOLO11 模型实时识别钢球，经标定得到位置与速度信息；控制端融合位置误差与速度前馈构成预测误差，有效补偿延迟，提升动态稳定性；同时支持 WebRTC 无线图传与同步录屏。

选用方案二。

### 3. 水管倾角执行机构选型

方案一：舵机驱动。控制简单、体积小，但齿轮回差大，高频微调下重复定位性差，难以满足高精度控制需求。

方案二：闭环步进电机+偏心连杆驱动。X42S 驱动器驱动 42 型闭环步进电机，经偏心连杆转换为水管俯仰，调节范围 $\pm 50^\circ$ 。电机自带 1000 线编码器实现全闭环控制，无丢步，重复定位精度高；通过串口接收角度指令，调试便捷、可靠性强。

选用方案二。

## 四、硬件系统设计

### 1. 系统硬件总体连接

系统硬件整体连接关系如图 2 所示，以 MSPM0G3507 为主控核心，分为三大硬件支路：底盘支路由 8 路灰度传感器、双 GMR 编码器采集循线与速度信息，经主控运算后通过 TB6612 驱动 MG513 电机；执行支路由 MaixCAM2 输出钢球位置与速度数据，主控输出倾角指令经 X42S 驱动闭环步进电机，调节水管俯仰角度；配套 OLED 显示模块与电源模块，分别实现状态交互与分级供电。

### 2. MSPM0G3507 主控最小系统

主主控最小系统以 MSPM0G3507 芯片为核心，配备 3.3V 稳压、8MHz 外部晶振、上电复位、SWD 调试接口及电源去耦电路。系统由 5V 经 LD0 稳压至 3.3V 供电，VDD 端并联 100nF 与 10  $\mu$ F 电容抑制电源纹波。BOOT0 引脚接地，配置芯片从 Flash 启动。主控通过 GPIO、定时器捕获、UART 及 I2C 外设接口，分别对接底盘传感、执行机构与显示模块。

### 3. 循线底盘模块

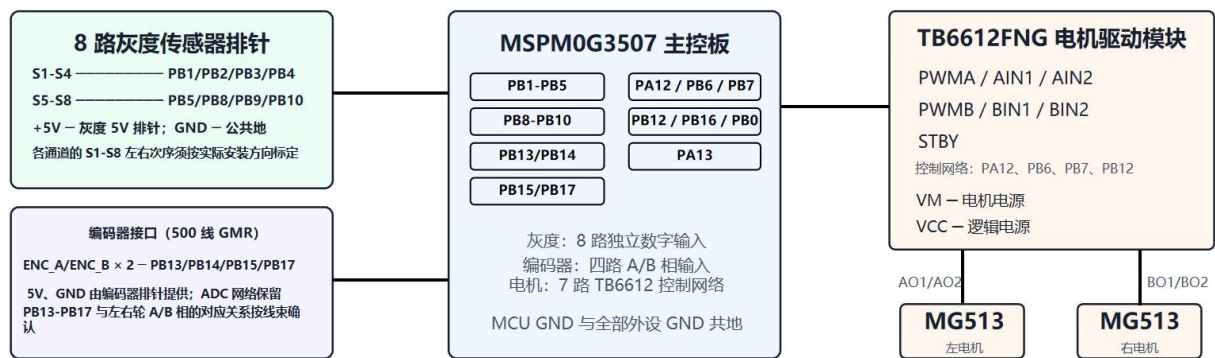


图 3 循线底盘模块接口电路示意图

循线底盘模块接口电路如图 3 所示，包含电机驱动与传感反馈两部分。电机驱动采用 TB6612FNG 双通道直流电机驱动芯片，VM 引脚接 12V 动力电源，VCC 引脚接 3.3V 逻辑电源；AIN1/AIN2、BIN1/BIN2 引脚接收主控方向与制动控制信号，PWMA/PWMB 引脚接收 PWM 调速信号，STBY 引脚由 GPIO 控制芯片使能。后端驱动 MG513 直流减速电机，电机电源端并联  $100\mu\text{F}$  与  $0.1\mu\text{F}$  电容，抑制电机换向产生的电磁干扰。传感反馈部分，8 路灰度传感器横向布置于车体前端，用于检测黑线偏移量，各通道按位置加权计算循线偏差；双路 500 线 GMR 编码器的 A/B 相接入主控定时器捕获端口，经四倍频后解算左右轮实时转速与累计里程，为速度闭环、分段减速触发与定点停车提供反馈依据。

#### 4. 视觉检测与水管倾角执行模块

视觉检测与倾角执行模块接口如图 4 所示，分为视觉感知与角度执行两部分。视觉感知端，MaixCAM2 固定于水管凹槽正上方，内置 YOLO11 模型识别钢球目标，通过检测框中心坐标与预标定的像素 - 距离映射关系，解算钢球实际位置与运动速度，以约 50fps 帧率经 UART 串口发送至主控；同时设备通过板载 WiFi，以 WebRTC 协议将

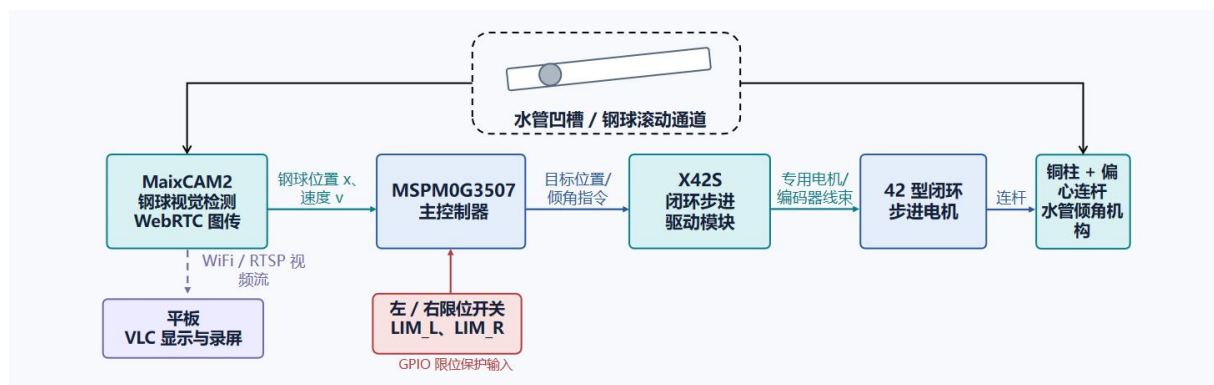


图 4 视觉检测与水管倾角执行模块接口图

实时画面推送至赛道外平板，支持浏览器端实时显示与同步录屏。角度执行端，主控通过 UART1 向 X42S 闭环步进驱动器发送倾角指令，驱动器驱动 42 型闭环步进电机运转，经偏心连杆与合页机构将旋转运动转换为水管俯仰动作，角度调节范围  $\pm 50^\circ$ ；驱动器依托电机自带编码器实现位置全闭环，无丢步问题。软件层对目标倾角做  $\pm 50^\circ$  限幅处理，视觉目标丢失或通信异常时自动输出零角度指令，保障系统运行安全。

## 5. 电源与人机交互模块

12V 锂电池为总电源，12V 直供 TB6612 与 X42S；低压支路生成 5V 供灰度传感器，5V 经 LD0 降至 3.3V 供主控与 OLED；MaixCAM2 自带电源独立供电。OLED 显示模式、计时与状态；按键用于模式选择与启停控制。

## 五、软件系统设计

### 1. 软件总体设计

系统软件基于 MSPM0G3507 平台采用模块化设计，以“主循环调度 + 固定周期闭环”为核心架构，包含系统初始化、传感采集、运动控制、通信交互、状态显示与异常保护六大功能模块。主循环负责按键模式管理与全局任务调度，底盘循线控制、钢球平衡控制按独立固定周期执行，保障控制实时性与逻辑稳定性。

循线行驶控制链路中，主控周期采集灰度传感器数据计算赛道偏移量，结合编码器反馈的转速与累计里程，通过差速 PID 算法输出左右电机 PWM 控制量。当编码器累计里程达到减速阈值时，小车切换低速模式搜索停车线；当不少于 4 路灰度传感器同时检测到黑线时，触发制动并保持停车状态。

钢球视觉控制链路中，MaixCAM2 通过串口向主控传输钢球位置、速度及目标检测标志，同步通过 WebRTC 协议向平板端推送实时画面。主控以 20ms 为固定周期解算预测误差，经 PID 运算输出目标倾角指令至 X42S 驱动器。系统设置多重异常保护：视觉目标丢失时立即清零控制状态，水管回归零角度；通信超时或角度超限时限制执行机构输出幅值，避免失控风险。

### 5.2 行驶过程钢球协同平衡控制

行驶工况下采用“底盘平稳循线 + 钢球主动平衡”的协同控制方案，从源头削弱惯性扰动，配合自适应算法提升全行程平衡稳定性。底盘侧通过软启动与分段减速策略，降低起步、转弯、停车阶段的惯性冲击。起步阶段基础速度从初始值按固定步长逐步提升，避免加速度突变；行驶过程中依据编码器

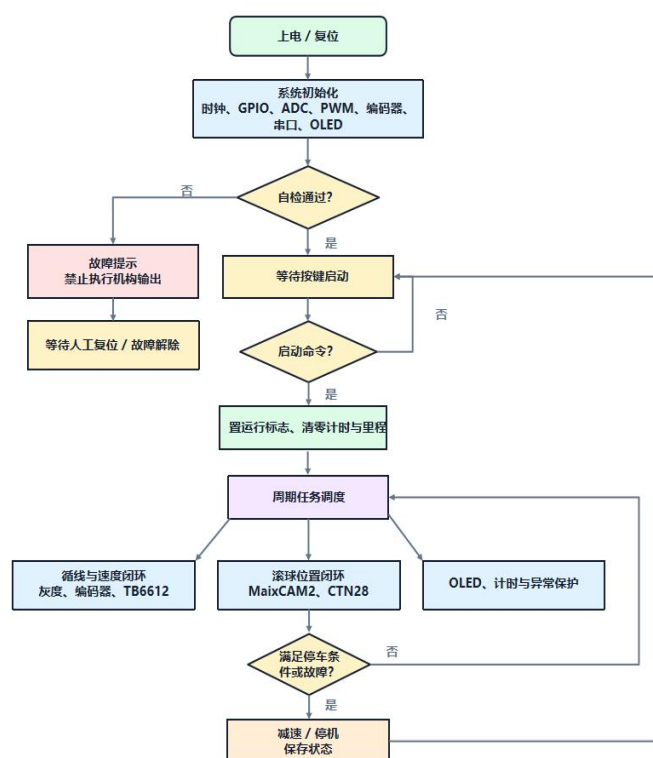


图 5 主程序流程图

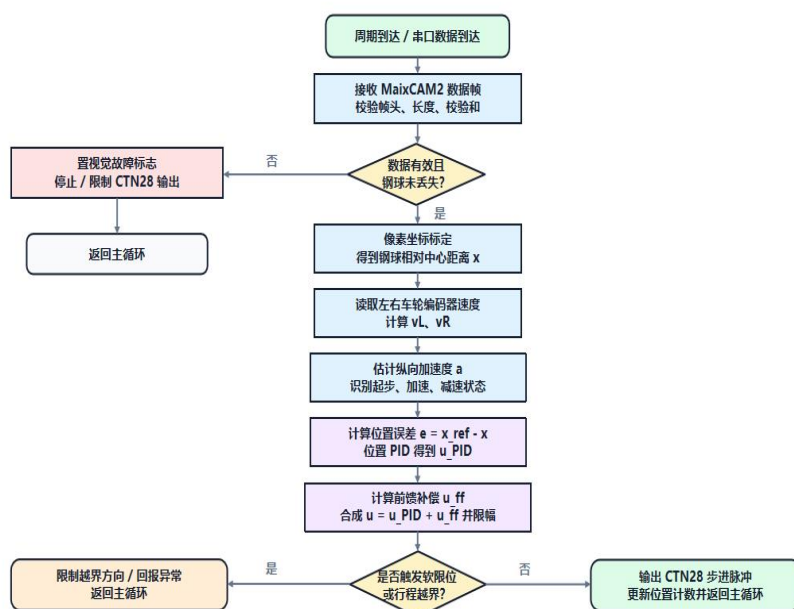


图 6 钢球视觉位置闭环与 X42S 控制子程序流程图

里程识别路段位置，接近减速区时逐级下调目标速度；停车采用“里程预判 + 灰度识别”双判据触发，当累计里程达到终点阈值，或不少于 4 路灰度传感器同时检测到停车黑线时，执行平稳制动。

钢球控制侧沿用预测误差 PID 算法，并针对加速、匀速、减速不同阶段的扰动特性动态调整控制增益。匀速阶段采用基准增益保证稳态定位精度；加减速阶段适配调整比例与微分系数，主动抵消惯性扰动带来的位置偏移，实现全行驶工况下钢球的稳定控制。

### 5.3 循线与速度闭环控制

灰度传感器各通道按横向位置赋权计算循线偏差，依据偏差生成左右轮目标速度差；编码器实测转速构成速度反馈，采用增量式 PID 调整左右电机 PWM 输出。丢线时保持上一时刻偏差，以减小突变控制量。

$$e(k) = x(k) - x_{ref}$$

循线偏差：

$$e_{line} = \frac{\sum (w_i g_i)}{\sum g_i}$$

，其中  $w_i$  为通道权重， $g_i$  为通道响应，丢线时保持上一时刻值。双轮差速：

$$v_{Lref} = v_{ref} - \Delta v$$

$$v_{Rref} = v_{ref} + \Delta v$$

，其中  $\Delta v$  由循线偏差经比例控制产生，编码器实测转速反馈经增量式 PID 完成速度内环闭环。

### 5.4 钢球预测误差 PID 控制

控制器将钢球位置误差与速度提前项合成为预测误差，并以 20 ms 周期输水管目标倾角。当前位置采用比例控制为主，目标角度限制在  $\pm 50^\circ$ ；视觉目标丢失时清零控制状态并使水管回归零角度。

钢球预测误差 PID:  $e_{pre} = e + k_v v$ ，其中  $e = x - x_{ref}$  为位置误差， $v$  为钢球实时速度， $k_v$  为视觉测得钢球速度， $k_v = 0.5$  为速度提前系数。 $u = -(K_p e_{pre} + K_i \sum e + K_d \Delta e_{pre})$

，当前取  $K_p = 5.0$ ， $K_i = K_d = 0$ 。输出限幅  $\pm 50^\circ$  后经 RS485 发送至 X42S。视觉丢失时积分项与预测误差清零，目标角度置零。控制周期 20 ms。

## 六、测试方案与测试结果

### 1、测试方案

#### (1) 关键硬件模块测试

完成了 8 路灰度循线传感器的黑白识别阈值标定，测试了不同安装高度下的响应灵敏度，验证了信号跳变清晰度与抗干扰能力；验证了 GMR 编码器的测速精度与线性度；测试了 TB6612 对 MG513 电机的驱动能力与响应速度；完成了 MaixCAM2 视觉模块的像素-距离映射标定，验证了 WebRTC 推流、平板端实时显示及录屏回放功能的稳定性。

#### (2) 控制算法与软件功能测试

对循线 PID 和速度 PID 进行了参数整定，通过电机阶跃响应确定最佳参数。针对滚

球预测误差 PID，先整定比例系数  $K_p$  和速度提前系数  $k_v$ ，依据稳态偏差和振荡情况确定  $K_p=5.0$ 、 $k_v=0.5$ ，并加入目标角度限幅，实现快速响应与稳定控制。

### （3）整车联调测试

组装完成后在规定赛道上进行系统联调，依次完成小车循线、定点停车、静态往返滚球控制、短距动态平衡及整圈任意点动态平衡等多项指标的重复测试，每次测试均记录视频编号与原始数据。

## 2、测试条件与仪器

按题目要求布设环形赛道和 A、B、C、D 点。测试前检查钢球滚动流畅度、摄像头覆盖范围、灰度传感器安装高度和黑白标定状态。

采用直尺、系统计时、万用表、示波器和具备录屏功能的平板进行测试。每项指标重复 10 次，记录平均值、最大误差及异常情况。

## 3、测试结果及分析

### （1）测试结果

详细统计数据及原始记录见附录 2，正文仅总结关键指标与结论。

### （2）测试分析与结论

图传与录像：WebRTC 实时显示清晰流畅，录屏文件完整保存了钢球运动视频并支持回放，图传连续无断联。

循线与停车：小车沿黑线行驶一圈停至 A 点，平均耗时 18.35s，最大停车偏差 1.7cm，满足要求。

静态滚球控制：小球从中心点往  $\pm 5\text{cm}$  处折返并稳定，最长耗时 3.83s，最大误差 0.9cm，满足要求。

动态滚球控制：AB 段最大耗时 6.91s；整圈中心点平衡最大耗时 24.90s；整圈任意指定位置平衡最大耗时 25.12s，最大误差均为 0.9cm，满足要求。

测试结果表明，系统循线平稳，预测误差 PID 算法有效缓解了视觉滞后与行驶扰动，图传、运动时间及钢球平衡误差均满足赛题要求。

## 七、结束语

本系统以 MSPM0G3507 为核心，完成了车载平衡滚球运动控制系统的设计与制作，可实现环形循线行驶、钢球静态定位及动态平衡、WebRTC 图传录屏等主要项功能。

系统经多方案对比论证完成器件选型，按主控向外围的思路逐模块完成软硬件设计，采用预测误差 PID 提升钢球控制响应速度。后续可进一步整定控制参数、补充 IMU 辅助反馈、优化机械连杆结构，提升系统稳定性与精度。

## 八、参考文献

[1] 全国大学生电子设计竞赛组委会. 2026 年全国大学生电子设计竞赛赛题：车载平衡滚球运动控制系统（H 题）[Z].

[2] Texas Instruments. MSPM0G350x Mixed-Signal Microcontrollers Technical Reference Manual[Z].

[3] Texas Instruments. TB6612FNG Dual DC Motor Driver Datasheet[Z].

[4] Sipeed. MaixCAM2 Documentation[Z].

[5] 感为科技. 感为无 MCU 八路灰度传感器手册[Z].

[6] X42S 闭环步进驱动模块使用说明书[Z].

[7] 42 型闭环步进电机使用说明书[Z].

[8] MG513 有刷直流减速电机产品手册[Z].

[9] 500 线 GMR 编码器使用说明书[Z].

附录 1： 各模块硬件接口引脚分配图

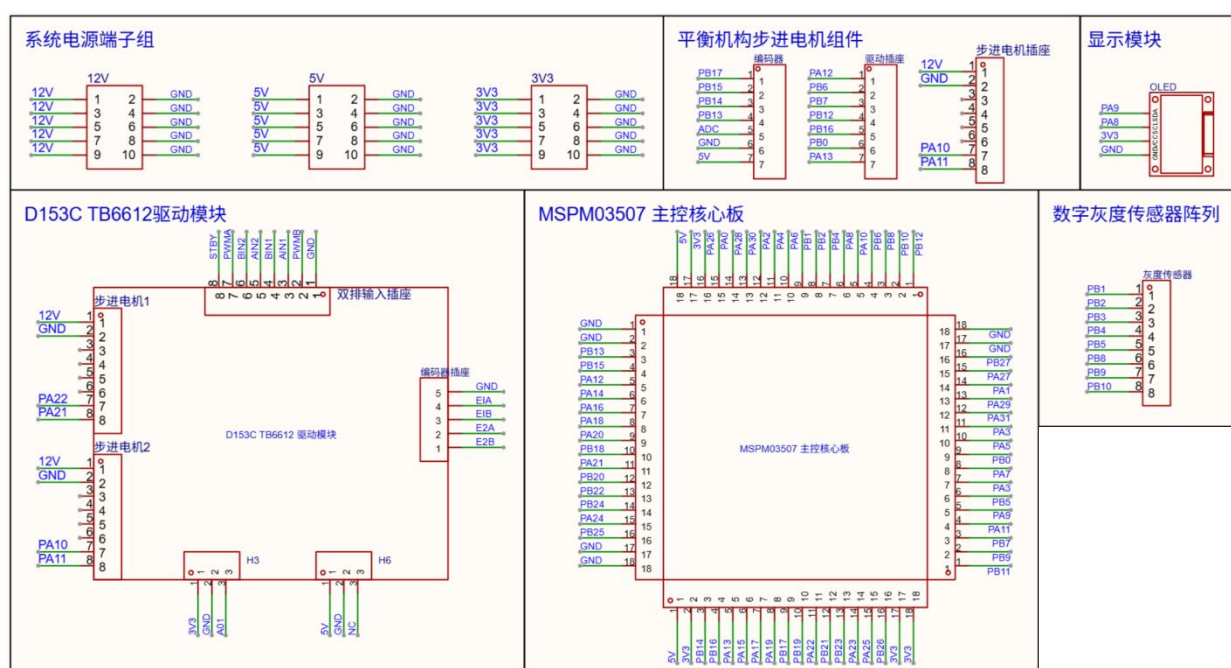


图 7 系统硬件接口分配框图

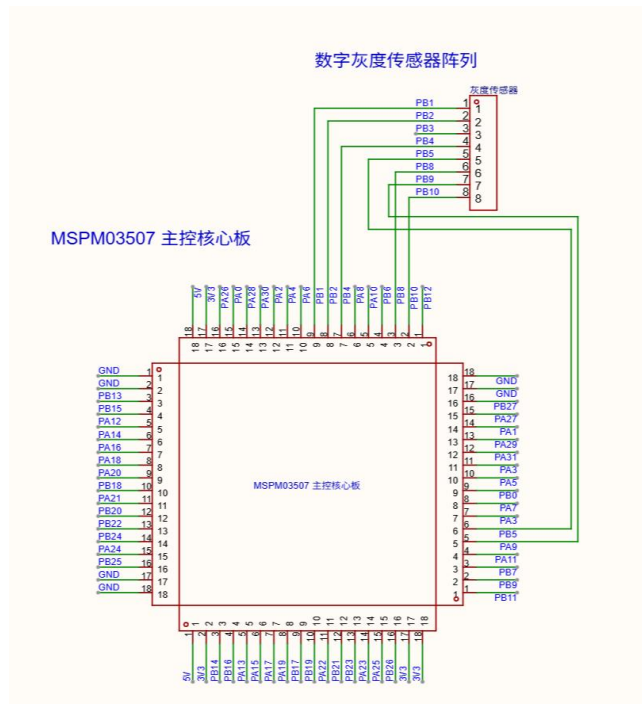


图 8 MSPM03507 主控与数字灰度传感器阵列硬件连接示意图

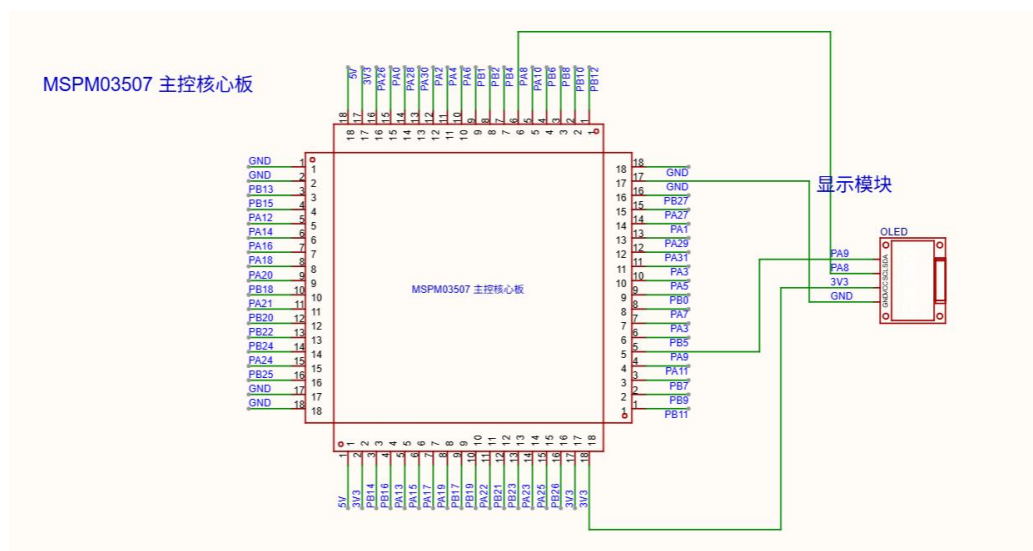


图 9 MSPM03507 主控与 OLED 显示模块硬件连接示意图

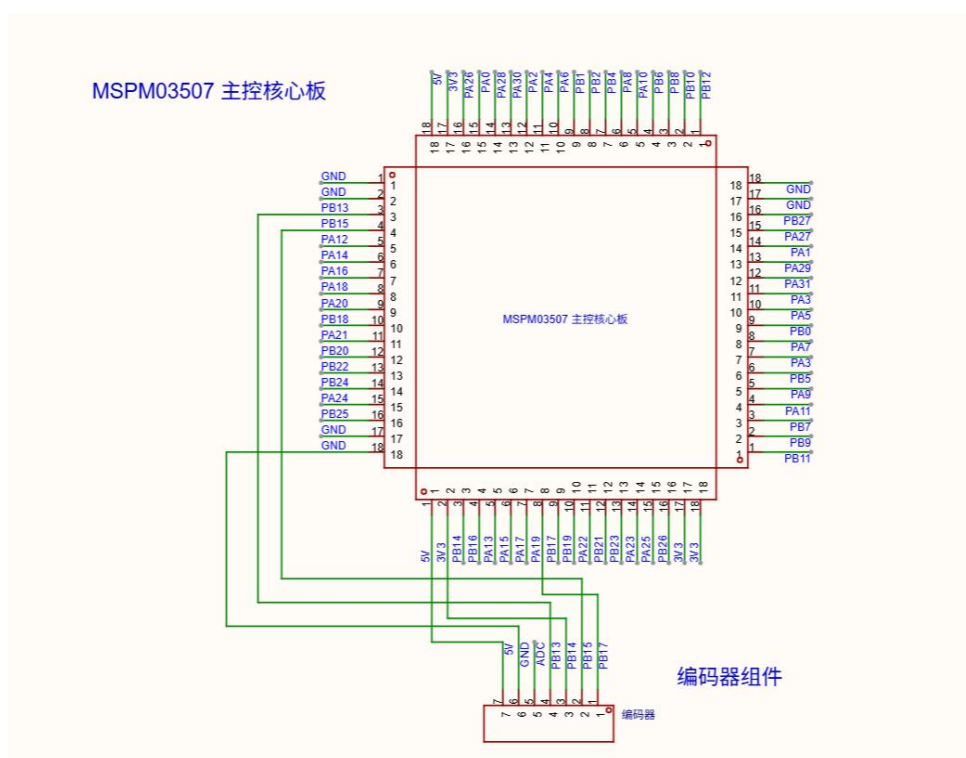


图 10 MSPM03507 主控与编码器组件接口连接示意图

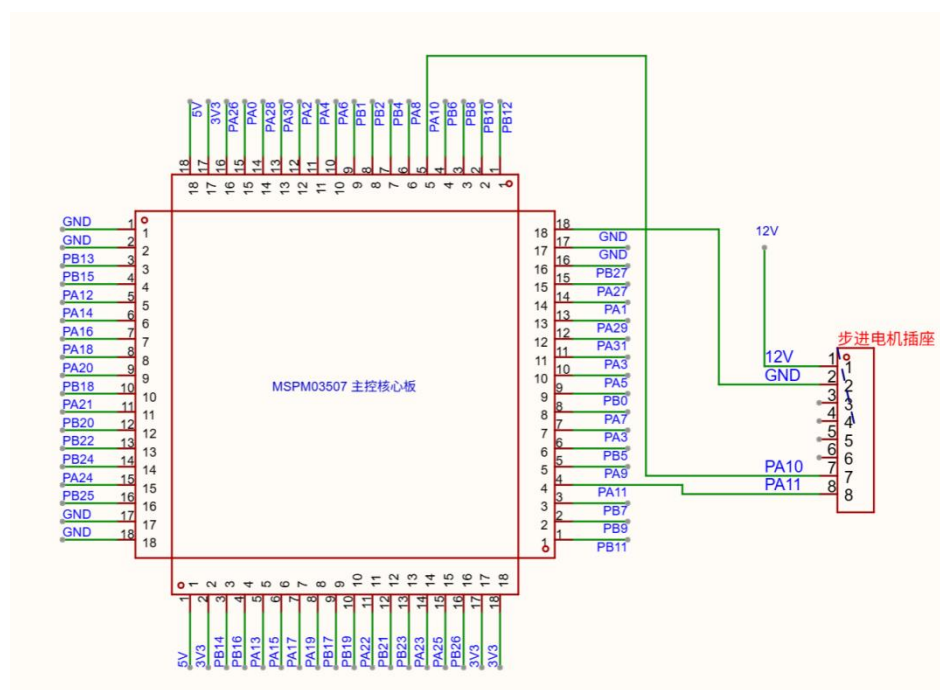


图 11 MSPM03507 主控与步进电机硬件连接示意图

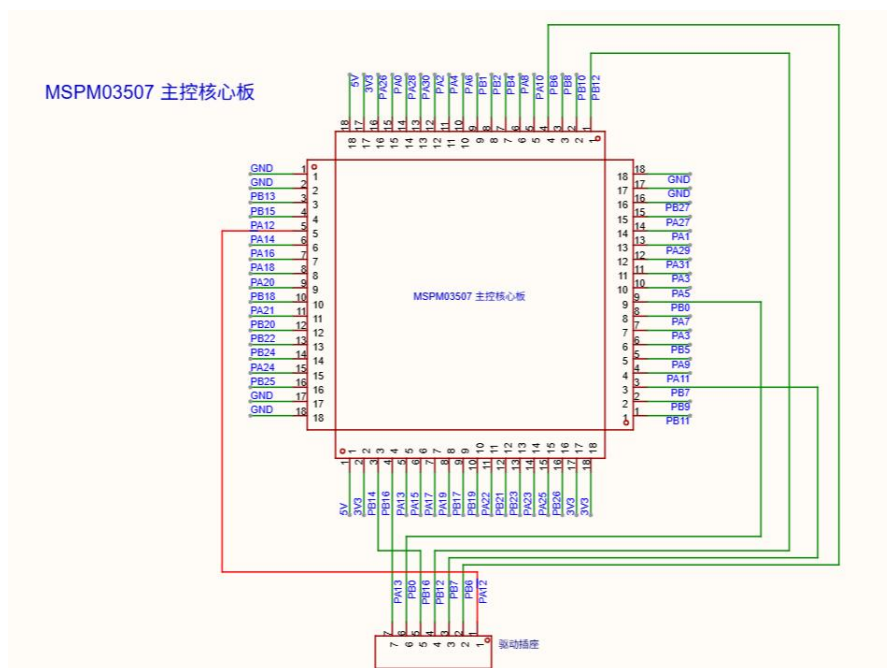


图 12 MSPM03507 主控与 D153C 驱动模块接口连接示意图

## 附录 2: 测试原始数据与统计表

表 A-1 小车整圈运行与停车测试结果

| 测试序号 | 一圈时间 (s) | 停车偏差 (cm) |
|------|----------|-----------|
| 1    | 18.42    | 1.05      |
| 2    | 18.87    | 0.92      |
| 3    | 18.15    | 1.18      |
| 4    | 18.63    | 0.85      |
| 5    | 18.91    | 1.02      |
| 6    | 18.28    | 1.11      |
| 7    | 18.75    | 0.98      |
| 8    | 18.54    | 1.09      |
| 9    | 18.36    | 0.88      |
| 10   | 18.82    | 1.15      |

表 A-2 钢球往返定位测试结果

| 测试序号 | 误差 (cm) | 时间 (s) |
|------|---------|--------|
| 1    | 0.8     | 3.55   |
| 2    | 0.2     | 3.22   |

|    |     |      |
|----|-----|------|
| 3  | 0.5 | 3.03 |
| 4  | 0.9 | 3.77 |
| 5  | 0.8 | 3.66 |
| 6  | 0.7 | 3.34 |
| 7  | 0.7 | 3.54 |
| 8  | 0.6 | 3.4  |
| 9  | 0.9 | 3.83 |
| 10 | 0.4 | 3.45 |

表 A-3 AB 段行驶过程钢球平衡测试结果

| 测试序号 | 误差 (cm) | 时间 (s) |
|------|---------|--------|
| 1    | 0.3     | 6.23   |
| 2    | 0.6     | 6.85   |
| 3    | 0.8     | 6.45   |
| 4    | 0.4     | 6.12   |
| 5    | 0.9     | 6.91   |
| 6    | 0.5     | 6.55   |
| 7    | 0.2     | 6.34   |
| 8    | 0.7     | 6.78   |
| 9    | 0.5     | 6.6    |
| 10   | 0.8     | 6.42   |

表 A-4 整圈运行钢球平衡测试结果

| 测试序号 | 误差 (cm) | 时间 (s) |
|------|---------|--------|
| 1    | 0.5     | 23.45  |
| 2    | 0.8     | 24.12  |
| 3    | 0.3     | 23.88  |
| 4    | 0.7     | 24.65  |
| 5    | 0.6     | 23.15  |

|    |     |       |
|----|-----|-------|
| 6  | 0.9 | 24.9  |
| 7  | 0.4 | 23.77 |
| 8  | 0.8 | 24.34 |
| 9  | 0.2 | 23.56 |
| 10 | 0.5 | 24.05 |

表 A-5 指定位置钢球定位测试结果

| 测试序号 | 指定位置 (cm) | 误差 (cm) | 时间 (s) |
|------|-----------|---------|--------|
| 1    | +4.0      | 0.6     | 24.11  |
| 2    | -6.0      | 0.3     | 23.58  |
| 3    | +8.5      | 0.7     | 24.75  |
| 4    | -3.5      | 0.5     | 23.9   |
| 5    | +10.0     | 0.9     | 25.12  |
| 6    | -8.0      | 0.4     | 23.45  |
| 7    | +2.0      | 0.2     | 24.3   |
| 8    | -10.0     | 0.8     | 24.88  |
| 9    | +6.5      | 0.5     | 23.77  |
| 10   | -4.0      | 0.7     | 24.05  |
| 测试序号 | 指定位置 (cm) | 误差 (cm) | 时间 (s) |
| 1    | +4.0      | 0.6     | 24.11  |

### 附录 3：源程序

```
// ===== 主函数 (empty.c) =====
int main(void)
{
    SYSCFG_DL_init();
    motor_init(); track_init(); GetYaw_Init();
    Encoder_Init(); SoundLight_Init(); Camera_Init();
}
```

```

Stepper_Init(); delay_ms(200); Stepper_SetZero();
OLED_Init(); delay_ms(20); GetYaw_Reset();

uint8_t mode = 0, running = 0;
while (1) {
    Camera_Task();
    // K1: 菜单切换 mode
    if ((running == 0U) && (!DL_GPIO_readPins(GPIO_KEY_PORT,
GPIO_KEY_K1_PIN))) {
        delay_ms(20);
        if (!DL_GPIO_readPins(GPIO_KEY_PORT, GPIO_KEY_K1_PIN)) {
            mode = (uint8_t)((mode + 1U) % 5U);
            while (!DL_GPIO_readPins(GPIO_KEY_PORT, GPIO_KEY_K1_PIN))
delay_ms(10);
        }
    }
    // K2: 确认启动
    if ((running == 0U) && (mode != 0U) &&
        (!DL_GPIO_readPins(GPIO_KEY_PORT, GPIO_KEY_K2_PIN))) {
        delay_ms(20);
        if (!DL_GPIO_readPins(GPIO_KEY_PORT, GPIO_KEY_K2_PIN)) {
            motors_stop(); delay_ms(1000);
            SoundLight_TimeStart(); GetYaw_Reset();
            track_reset(mode); running = 1U;
            while (!DL_GPIO_readPins(GPIO_KEY_PORT, GPIO_KEY_K2_PIN))
delay_ms(10);
        }
    }
    // K3: 调试启动
    if ((running == 0U) && (!DL_GPIO_readPins(GPIO_KEY_PORT,
GPIO_KEY_K3_PIN))) {
        delay_ms(20);
        if (!DL_GPIO_readPins(GPIO_KEY_PORT, GPIO_KEY_K3_PIN)) {
            motors_stop(); delay_ms(500);
            SoundLight_TimeStart(); GetYaw_Reset();
            mode = 6U; track_reset(mode); running = 1U;
            while (!DL_GPIO_readPins(GPIO_KEY_PORT, GPIO_KEY_K3_PIN))
delay_ms(10);
        }
    }

    if (running == 0U) { motors_stop(); }
    else {

```

```

        switch (mode) {
            case 1: t2();                break;    // 纯循线（略，见 t4 下
半部分）
            case 2: t3();                break;    // 钢球往返定位
            case 3: t4_weizhi();          break;    // 短距行驶+平衡
            case 4: t5_weizhi();          break;    // 整圈行驶+平衡（同 t4
结构）
            case 6: t3_yuce();            break;    // 调试
            default: motors_stop(); running = 0U; break;
        }
    }
}

```

// ===== 钢球往返定位 (track.c) =====

```

void t3(void)
{
    static uint8_t state = 0U;           // 0→+5cm, 1→-5cm, 2→稳定
    static float pos_integral = 0.0f;
    static float last_predict_error = 0.0f;
    static uint32_t last_stepper_time = 0U;

    uint32_t now_time = SoundLight_GetSysMs();
    if ((now_time - last_stepper_time) < 20U) return;
    last_stepper_time = now_time;

    if (Camera_HasBall() == 0U) {
        pos_integral = 0.0f; last_predict_error = 0.0f;
        Stepper_SetAngle(0.0f); return;
    }

    float ball_pos    = Camera_GetBallError();
    float ball_speed   = Camera_GetBallSpeed();
    float target_pos   = (state == 0U) ? 5.0f : -5.0f;
    float ball_error    = ball_pos - target_pos;

    // 到达目标→切换状态
    if (ball_error <= 0.5f && ball_error >= -0.5f) {
        state = (state == 0U) ? 1U : 2U;
        pos_integral = 0.0f; last_predict_error = 0.0f;
    }
}

```

```

float kv = (state == 1U) ? 0.78f : 0.5f;    // 折返段加大速度提前
float kp = 5.0f, ki = 0.0f, kd = 0.05f;
float predict_error = ball_error + kv * ball_speed;

pos_integral += ball_error;
if (pos_integral > 800.0f) pos_integral = 800.0f;
if (pos_integral < -800.0f) pos_integral = -800.0f;

float rod_angle = -(kp * predict_error + ki * pos_integral
                    + kd * (predict_error - last_predict_error));
if (rod_angle > 50.0f) rod_angle = 50.0f;
if (rod_angle < -50.0f) rod_angle = -50.0f;

last_predict_error = predict_error;
Stepper_SetAngle(rod_angle);
}

```

```

// 短距行驶+钢球平衡 (track.c) — 钢球控制部分
// 循线底盘部分 (灰度差速+编码器速度 PID) 与 t2 结构相同, 此处省略
void t4_weizhi(void)
{

```

```

    // --- 状态与参数初始化 (略) ---
    // state: 0 加速 / 1 匀速 / 2 减速
    // soft_reset: 软启动标志
    // ...[初始化代码见源文件 track.c L2001-L2214]...

/* ===== 钢球预测误差 PID 控制 (20ms 周期) ===== */
    if (Camera_HasBall() == 0U) {
        pos_integral = 0.0f; last_predict_error = 0.0f;
        Stepper_SetAngle(0.0f);
    } else {
        float kv = 0.53f, kp = 5.0f;
        if (state == 0U) kp = 6.5f;    // 加速段加大 P
        else if (state == 2U) kp = 6.0f;    // 减速段加大 P

        float ball_pos = Camera_GetBallError();
        float ball_speed = Camera_GetBallSpeed();
        float predict_error = (ball_pos - 0.0f) + kv * ball_speed;

        pos_integral += (ball_pos - 0.0f);
        if (pos_integral > 800.0f) pos_integral = 800.0f;
        if (pos_integral < -800.0f) pos_integral = -800.0f;
    }
}

```

```

        float rod_angle = -(kp * predict_error + 0.0f * pos_integral
                               + 0.0f * (predict_error - last_predict_error));
        if (rod_angle > 50.0f) rod_angle = 50.0f;
        if (rod_angle < -50.0f) rod_angle = -50.0f;
        last_predict_error = predict_error;
        Stepper_SetAngle(rod_angle);
    }

    /* ===== 灰度循线 + 编码器速度闭环（与 t2 结构相同） ===== */
    // 灰度检测→差速计算→左右轮目标速度→增量式PID→motorA/B 输出
    // ...[完整代码见源文件 track.c L521-L630]...
    // 关键参数：编码器比=6, kp=0.2, ki=0.01, 软启动 5→250, 减速阈值
150000
    }

    // ===== 视觉核心代码 (main.py) =====

    class AdaptiveAlphaBetaFilter:
        """自适应位置+速度滤波器"""
        def __init__(self, alpha_slow=0.20, alpha_fast=0.95,
                      beta_slow=0.03, beta_fast=0.35, fast_error_px=8.0,
reset_ms=250):
            self.alpha_slow = alpha_slow; self.alpha_fast = alpha_fast
            self.beta_slow = beta_slow; self.beta_fast = beta_fast
            self.fast_error_px = fast_error_px; self.reset_ms = reset_ms
            self.reset()

        def reset(self):
            self.x = self.y = None; self.vx = self.vy = 0.0
            self.last_ms = self.last_meas_x = self.last_meas_y = None

        def update(self, measured_x, measured_y, now_ms):
            if self.x is None: # 首次初始化
                self.x = self.last_meas_x = float(measured_x)
                self.y = self.last_meas_y = float(measured_y)
                self.last_ms = now_ms; return self.x, self.y

            dt = max(0.005, min(0.100, (now_ms - self.last_ms) * 0.001))
            vx_direct = (measured_x - self.last_meas_x) / dt
            self.last_meas_x, self.last_meas_y = float(measured_x),
float(measured_y)

```

```

        predicted_x = self.x + self.vx * dt
        error_x = measured_x - predicted_x
        error = math.hypot(error_x, measured_y - (self.y + self.vy * dt))

        motion = min(1.0, error / self.fast_error_px) ** 2
        alpha = self.alpha_slow + (self.alpha_fast - self.alpha_slow) *
motion
        beta = self.beta_slow + (self.beta_fast - self.beta_slow) *
motion

        self.x = predicted_x + alpha * error_x
        self.y = self.y + self.vy * dt + alpha * (measured_y - (self.y +
self.vy * dt))
        gamma = 0.12
        self.vx = (1 - gamma) * (self.vx + beta * error_x / dt) + gamma *
vx_direct
        self.last_ms = now_ms
        return self.x, self.y

    def mark_missing(self, now_ms):
        if self.last_ms is not None and now_ms - self.last_ms >=
self.reset_ms:
            self.reset()

def serial_send_ball(valid, pos_cm, vx, confidence, timestamp_ms):
    """向 MSPMOG3507 发送钢球数据, 协议: $BALL, valid, cm, vx, conf, ms*XX"""
    global _serial_last_send_ms, serial
    if not serial: return
    if timestamp_ms - _serial_last_send_ms < 20: return // 20ms 间隔
    _serial_last_send_ms = timestamp_ms

    payload = "$BALL, {}, {:.2f}, {:.1f}, {:.2f}, {}".format(
        int(valid), pos_cm, vx, confidence, timestamp_ms)
    checksum = 0
    for ch in payload[1:]: checksum ^= ord(ch)
    packet = "{}*{:X}\n".format(payload, checksum)
    serial.write(packet.encode())

// ===== 视觉主循环 =====
AXIS_START_CM, AXIS_END_CM = -12.5, 12.5 // 水管有效行程

```

```

detector = nn.YOLO11(model="model_7263.mud") // YOLO11 检测模型
cam = camera.Camera(640, 480, image.Format.FMT_RGB888)
position_filter = AdaptiveAlphaBetaFilter()

while not app.need_exit():
    img = cam.read()
    now_ms = int(time.time() * 1000)

    // ROI 内筛选最高置信度钢球
    objs = detector.detect(img, conf_th=0.5, iou_th=0.45)
    roi_objs = [o for o in objs if roi_x <= o.x+o.w/2 <= roi_x+roi_w
                                     and roi_y <= o.y+o.h/2 <= roi_y+roi_h]
    ball = max(roi_objs, key=lambda o: o.score) if roi_objs else None

    if ball is not None:
        raw_x, raw_y = ball.x + ball.w*0.5, ball.y + ball.h*0.5
        filtered_x, filtered_y = position_filter.update(raw_x, raw_y,
now_ms)
        position_cm, _, _ = position_from_pixel(
            (filtered_x, filtered_y), AXIS_START_PX, AXIS_END_PX,
            AXIS_START_CM, AXIS_END_CM)
        if position_cm < 0: position_cm *= 1.11 // 负半轴修正系数
        vx_cms = position_filter.vx * 25.0 / roi_w // px/s→cm/s
        serial_send_ball(1, position_cm, vx_cms, ball.score, now_ms)
    else:
        position_filter.mark_missing(now_ms)
        serial_send_ball(0, 0.0, 0.0, 0.0, now_ms)

```